Microsoft Access Class Objects - Changes I Would Make

Focus: class objects only. Modules still need to be reviewed later.

Main changes I would start with

- **1. ModelBuildForm:** Break it up first. This is where most of the risk is. It is handling the form, build setup, snapshot updates, queue setup, output naming, and calls to ModelWriter/AuxWriter all in one place.
- **2. SQL execution:** Stop using repeated DoCmd.SetWarnings False + DoCmd.RunSQL for update/insert logic. Use a small helper around CurrentDb.Execute so errors are caught instead of hidden.
- **3. Snapshot updates:** The snapshot update logic is repeated in several places. Make one shared routine for this instead of copying the same update block in the single build path, queued build path, setLoadValue, and CloseMessageBox reset.
- **4. Queue design:** The queue is built from many parallel arrays like OutputDirQ(9), SnapshotQ(9), SnapTypeQ(9), etc. This works, but it is easy to break when a new build option is added.
- **5. Basic VBA cleanup:** Add Option Explicit everywhere, remove SetFocus calls used only for reading values, rename generic controls when the form is touched, and clean up obvious unreachable code or weak null checks.

Where the changes are

File	What I noticed	What I would change	Priority
Form_ModelBuildForm	Big form does too much; repeated snapshot setup; huge parameter lists; queue arrays; weak error cleanup.	Split build setup into smaller routines; create shared snapshot helper; replace RunSQL blocks; redesign queue item storage.	High
Form_UsrLog_Form	Hard-coded users/password; full scan of tblUserLog; SetWarnings/RunSQL inserts and updates.	Move users/password control to table/config; query only active user; use Execute with error handling.	High
Form_DynamicsAddModelForm	Finds next ID by scanning table; uses = Null check; builds SQL strings directly; generic control names.	Use DMax or AutoNumber; use IsNull; parameterize or at least centralized Execute; rename controls as touched.	Medium
Form_CloseMessageBox	ResetSnapshotsTable duplicates snapshot reset logic and can delete snapshot rows while closing.	Reuse same snapshot reset helper used by the build form; avoid table deletes during close unless intentional.	Medium
Form_DynamicExportForm	Small wrapper form, but has typo OuputDir and generic control names.	Low-risk cleanup when touched.	Low
Form_BranchMainForm	Only calls ImpCalc.CalculateAllImpedances.	No major issue. Add Option Explicit and consider clearer button name.	Low
Form_ShowUsersForm	This one is cleaner and already uses Option Explicit.	Use this as the style example for the smaller forms.	Low

1. Add Option Explicit to the forms that do not have it

Where: Form_ModelBuildForm, Form_UsrLog_Form, Form_CloseMessageBox, Form_DynamicExportForm, Form_DynamicsAddModelForm, Form_BranchMainForm.

Change: Add Option Explicit directly under Option Compare Database, then compile the project and fix the variables it catches.

Why: This is one of the easiest wins. It catches misspelled variables and accidental undeclared variables. In ModelBuildForm there are variables used like SQL, index, SnapIntOR, Snapshot, and exitMsg in places where the declarations are not always obvious. A compile pass would force the code to be clearer.

Before pattern:

```
Option Compare Database
Dim QueueCount As Integer
Dim UseQueue As Boolean
```

After pattern:

```
Option Compare Database
Option Explicit

Private QueueCount As Integer
Private UseQueue As Boolean
```

This does not change behavior by itself, but it will expose places that need cleanup before the database can compile cleanly.

2. Replace SetWarnings False / RunSQL blocks

Where: Form_ModelBuildForm, Form_UsrLog_Form, Form_CloseMessageBox, Form_DynamicsAddModelForm.

Change: Use CurrentDb.Execute with dbFailOnError or a small helper routine. This keeps warnings from being left off and gives better error handling.

Why: A lot of the class-object code turns warnings off, runs SQL, and turns warnings back on. If the code exits early, the database can be left in a weird state. It also hides the actual error that caused the failure.

Before pattern:

```
DoCmd.SetWarnings False
SQL = "UPDATE Machines SET is_slack = false WHERE bus_number <> " & Me.slackBusNum.Value
DoCmd.RunSQL SQL
SQL = "UPDATE Machines SET is_slack = true WHERE bus_number = " & Me.slackBusNum.Value
DoCmd.RunSQL SQL
DoCmd.SetWarnings True
```

After pattern:

```
Private Sub ExecuteSql(ByVal sqlText As String)
    CurrentDb.Execute sqlText, dbFailOnError
End Sub

ExecuteSql "UPDATE Machines SET is_slack = False WHERE bus_number <> " & CLng(Me.slackBusNum.Value)
ExecuteSql "UPDATE Machines SET is_slack = True WHERE bus_number = " & CLng(Me.slackBusNum.Value)
```

This is not meant to say every query must be rewritten at once. I would start with the build path and the login path because those are most visible when something fails.

3. Stop repeating the same Snapshots update logic

Where: Form_ModelBuildForm and Form_CloseMessageBox.

Change: Create one shared snapshot update/reset routine and call it from the form events. Do not keep separate copies of the same update SQL in multiple places.

Why: The same kind of Snapshot update appears in btnBuildFiles, the queue build loop, setLoadValue, and ResetSnapshotsTable. This makes it easy for one path to get fixed while another path still has the old behavior.

Before pattern:

```
SQL = "UPDATE Snapshots SET [model_name] = '" & Right(Year(snapDate), 2) & snapSeason & "', [year] = '" &
Year(snapDate) & "', [season] = '" & snapSeason & "', [date] = '" _
& month(snapDate) & "/" & day(snapDate) & "/" & Year(snapDate) & "', [system_load_MW] = '" & SnapLoad & "',
[OR_interchange_MW] = '" & SnapIntOR & "', [study] = '" & SnapType _
& " " & Year(snapDate) & "', [psse_header1] = '" & Year(snapDate) & " " & SnapType & "', [psse_header2] = '" &
Year(snapDate) & snapSeason & " SCEG AREA'"
DoCmd.RunSQL SQL
```

After pattern:

```
Private Sub PrepareSnapshot(ByVal snapDate As Date, ByVal snapSeason As String, _
                            ByVal snapLoad As Long, ByVal snapType As String, _
                            ByVal snapIntOR As Variant)
    Dim modelName As String
    modelName = BuildSnapshotName(snapDate, snapSeason)

    ExecuteSql "UPDATE Snapshots SET " & _
               "model_name='" & modelName & "', " & _
               "[year]=" & Year(snapDate) & ", " & _
               "season='" & snapSeason & "', " & _
               "[date]=#" & Format$(snapDate, "yyyy-mm-dd") & "#, " & _
               "system_load_MW=" & CLng(snapLoad) & ", " & _
               "OR_interchange_MW=" & Nz(snapIntOR, 0) & ", " & _
               "study='" & snapType & " " & Year(snapDate) & "'"
End Sub
```

A better final version would use parameters instead of string concatenation, but even this helper shows the main idea: one place owns the snapshot update.

4. Avoid full-table recordset loops when the code only needs an update

Where: Form_ModelBuildForm, Form_CloseMessageBox, Form_UsrLog_Form, and Form_DynamicsAddModelForm.

Change: Use a direct query with a WHERE clause when the code is looking for one record or updating a known group of records.

Why: Several forms open a whole table, MoveLast, MoveFirst, loop through every row, and then run an UPDATE that affects the table. That is slower and makes the code harder to read. It also adds extra table churn to an Access file that already has bloat concerns.

Before pattern:

```
Set UserLog = dbUserLog.OpenRecordset("tblUserLog", dbOpenDynaset, dbSeeChanges)
UserLog.MoveLast
UserLog.MoveFirst

While Not UserLog.EOF
  If UserLog.Fields("UserID") = checkUser And IsNull(UserLog.Fields("LogOff")) Then
```

```
      MsgBox "Your user ID is currently logged in to this Database. Multiple Logins not allowed. Closing
Application"
      Application.Quit
  End If
  UserLog.MoveNext
Wend
```

After pattern:

```
If DCount("*", "tblUserLog", "UserID='" & Replace(checkUser, "'", "''") & "' AND LogOff Is Null") > 0 Then
    MsgBox "Your user ID is already logged into this database. Closing Application.", , "Multiple Logins Detected"
    Application.Quit
End If
```

For bigger tables, DCount may still not be the final answer. A parameterized QueryDef or indexed SELECT would be better. The point is that we do not need to scan the whole log table manually in the form.

5. Replace the queue arrays with one queue item design

Where: Form_ModelBuildForm.

Change: Instead of keeping 40+ separate arrays aligned by the same index, use a queue item class/type or store queued builds in a temporary table.

Why: The current queue design is fragile. Adding one new setting means adding a new array, setting it in SetQueuedVariables, reading it in the queue loop, and making sure the index always stays aligned.

Before pattern:

```
Dim OutputDirQ(9) As String
Dim FilenamePrefixQ(9) As String
Dim SnapshotQ(9) As Integer
Dim snapDateQ(9) As Date
Dim snapSeasonQ(9) As String
Dim SnapLoadQ(9) As Integer
Dim SnapTypeQ(9) As String
Dim RegionalQ(9) As Boolean
Dim busOffsetQ(9) As Long
' ...many more arrays...
```

After pattern:

```
Private Type BuildQueueItem
    OutputDir As String
    FilenamePrefix As String
    FilenameSuffix As String
    Snapshot As Long
    SnapDate As Date
    SnapSeason As String
    SnapLoad As Long
    SnapType As String
    Regional As Boolean
    BusOffset As Long
End Type

Private BuildQueue() As BuildQueueItem
Private QueueCount As Long
```

For Access, a temporary table may be even easier than a custom type if the queue needs to survive form refreshes or if the team wants to inspect queued builds.

6. Break btnBuildFiles into real stages

Where: Form_ModelBuildForm.

Change: Keep btnBuildFiles as the button event but move the actual work into smaller named routines. The form should read the screen, validate the request, prepare the snapshot, build the output path, run the writer, and reset the snapshot in separate steps.

Why: The button click is currently doing too many things. This makes it hard to tell where a failed build failed. Smaller routines would also make it easier to add logging around each stage.

Current active pattern, shortened:

```
Private Sub btnBuildFiles_Click()
    If Not UseQueue Then
        Set db = CurrentDb()
        Set Snapshots = db.OpenRecordset("Snapshots", dbOpenDynaset, dbSeeChanges)
        DoCmd.RunSQL "UPDATE Snapshots SET [Selected] = '-1'"
        DoCmd.RunSQL "UPDATE Machines SET is_slack = false WHERE bus_number <> " & Me.slackBusNum.Value

        If PSSESav Then
            ModelWriter.WriteOutFullModel FilePath, baseYear, Snapshot, Regional, busOffset, ...
        ElseIf PWaux Then
            AuxWriter.WriteOutFullModel FilePath, baseYear, Snapshot, Regional, busOffset, ...
        End If
    Else
        For i = 0 To QueueCount
            Set SnapshotsQ = dbQ.OpenRecordset("Snapshots", dbOpenDynaset, dbSeeChanges)
            DoCmd.RunSQL "UPDATE Snapshots SET [Selected] = '-1'"
            ModelWriter.WriteOutFullModel FilePathQ(counter), baseYearQ(counter), SnapshotQ(counter), ...
        Next
    End If
End Sub
```

Cleaner direction:

```
Private Sub btnBuildFiles_Click()
    On Error GoTo BuildFail

    Dim request As BuildRequest
    request = ReadBuildRequestFromForm()

    ValidateBuildRequest request
    PrepareSnapshot request
    UpdateSlackBus request.SlackBus
    request.FilePath = BuildOutputPath(request)
    RunModelOutput request
    ResetSnapshotToToday request

    MsgBox "Model Data Written to: " & request.FilePath
    Exit Sub

BuildFail:
    MsgBox "Build failed during setup: " & Err.Number & " - " & Err.Description
End Sub
```

The BuildRequest part would probably live in a standard module or class module. For this class-object section, the important takeaway is just that the form should stop holding the whole build process directly.

7. Fix the weak Null checks

Where: Form_DynamicsAddModelForm and any similar checks found later.

Change: Use IsNull() instead of = Null.

Why: In VBA, comparing with = Null does not work the way people expect. It does not reliably return True. That means the form could skip a validation that was intended to block missing values.

Before pattern:

```
If MachineID = Null Or ModelType = Null Then
    MsgBox "Cannot add model, Machine ID or Model Type cannot be null"
End If
```

After pattern:

```
If IsNull(Me.Combo20.Value) Or IsNull(Me.Combo22.Value) Then
    MsgBox "Cannot add model. Machine ID and Model Type are required."
    Exit Sub
End If

MachineID = CLng(Me.Combo20.Value)
ModelType = CLng(Me.Combo22.Value)
```

This is a small fix but worth doing because it is a real logic issue, not just style.

8. Stop using SetFocus just to read a control value

Where: Form_ModelBuildForm, Form_DynamicExportForm, Form_DynamicsAddModelForm.

Change: Read .Value directly when possible. Use .Text only when the control has focus and you specifically need the typed but unsaved text.

Why: A lot of the form code sets focus to a control before reading it. That makes the code noisier and can create odd UI side effects. Most of the time, .Value is what the code actually wants.

Before pattern:

```
Me.txtOutputDir.SetFocus
OutputDir = Me.txtOutputDir.Text

Me.txtFilenameSuffix.SetFocus
FilenameSuffix = Me.txtFilenameSuffix.Text
```

After pattern:

```
OutputDir = Nz(Me.txtOutputDir.Value, "")
FilenameSuffix = Nz(Me.txtFilenameSuffix.Value, "")
```

This is not likely to fix the build issue by itself, but it makes the code easier to read and safer to refactor.

9. Replace manual next-ID scans

Where: Form_DynamicsAddModelForm.

Change: Use an AutoNumber field if possible. If not, use DMax with error handling instead of opening the table and scanning every record.

Why: The current code opens DynamicsModels, moves through every row, finds the largest model_id, and adds 1. This can break if two users add at the same time, and it gets slower as the table grows.

Before pattern:

```
Set Models = db.OpenRecordset("DynamicsModels")
ModelID = 0
Models.MoveLast
Models.MoveFirst
While Not Models.EOF
  If Models.Fields("model_id").Value > ModelID Then
    ModelID = Models.Fields("model_id").Value
  End If
  Models.MoveNext
Wend
ModelID = ModelID + 1
```

After pattern:

```
ModelID = Nz(DMax("model_id", "DynamicsModels"), 0) + 1
```

DMax is still not perfect for multi-user writes. AutoNumber is the cleaner long-term fix if the table design allows it.

10. Move user access control out of VBA code (This one is optional)

Where: Form_UsrLog_Form.

Change: Now this one is optional but I think we should not keep the authorized user list and shared password hard coded in the form. Put authorized users in a table or controlled config.

Why: Right now, adding or removing access means editing code. The password is also visible in the VBA. This is not a huge corporate-security report, but it is still annoying maintenance-wise and easy to forget.

Extra note: This form looks like it stays open during the database session, probably as a startup or hidden session-control form. The code is not constantly running, but the form handles login, logout, edit-mode access, table hiding, and backups, so changes to this form should be tested carefully.

Before pattern:

```
Dim UserList(1 To 20) As String
UserList(1) = "au41679"
UserList(2) = "joh1375"
' ...
UserList(20) = "ningch1"

DB_password = "Planning"
```

After pattern:

```
AuthorizedUser = DCount("*", "AuthorizedUsers", _
    "UserID='" & Replace(sUser, "'", "''") & "' AND Active=True") > 0
```

Even a simple AuthorizedUsers table with UserID, DisplayName, Active, CanEdit, and Notes would be easier than editing VBA every time.

11. Add real error cleanup around build-related routines

Where: Form_ModelBuildForm and any form that writes tables.

Change: Use one error handler per important routine. Make sure objects close and warning state is restored if the old RunSQL pattern remains temporarily.

Why: The build code has early exits, On Error Resume Next, and a generic failure message. When a build stops halfway through, the team needs the step and actual error message, not just "something is broken."

Before pattern:

```
If AuxWriter.WriteOutFullModel(...) Then
    exitMsg = "Model Data Written to: " & FilePath
Else
    exitMsg = "Did not complete model write-out. Something is broken. Probably should let an expert know."
End If
```

After pattern:

```
If Not AuxWriter.WriteOutFullModel(...) Then
    Err.Raise vbObjectError + 2001, "btnBuildFiles", _
        "AuxWriter returned False for: " & FilePath
End If
```

Later, this should write to a build log table with build name, user, start time, stage, status, and error text.

12. Clean up small naming issues while touching each form

Where: Form_DynamicsAddModelForm, Form_DynamicExportForm, Form_BranchMainForm, and parts of Form_ModelBuildForm.

Change: Do not do a giant rename-only project. But when a form is already being edited, rename controls like Command18, Text31, and Text33 to names that explain what they are.

Why: Generic names make the code harder to review. A small team can live with old names, but it slows down anyone trying to trace the logic quickly.

Before pattern:

```
Private Sub Command18_Click() 'OK Button
Me.Text31.SetFocus
DateStart = Me.Text31.Text
Me.Text33.SetFocus
DateEnd = Me.Text33.Text
```

After pattern:

```
Private Sub btnAddModel_Click()
DateStart = Nz(Me.txtModelDateStart.Value, "")
DateEnd = Nz(Me.txtModelDateEnd.Value, "")
```

This is a low priority compared with build stability, but it is a good cleanup habit when editing a form anyway.

File-by-file notes

Form_ModelBuildForm

- Highest priority file in the class-object section.
- It has too many responsibilities in one form.
- The single-build and queue-build paths repeat the same ideas instead of sharing logic.
- Snapshot setup appears in more than one place.

- Queue arrays are the biggest maintainability smell in this file.
- Start here only after making a backup and after the team agrees on a test build case.

Form_UsrLog_Form

- Second highest priority because it runs when the database opens.
- The active-login check should not scan the whole log table.
- The authorized user list and password should not be hard-coded in the form.
- LogOn, LogOff, and FailedLogin should use Execute or a shared SQL helper.

Form_DynamicsAddModelForm

- Fix the = Null check first because that is a real logic issue.
- The manual next-ID logic should be replaced if possible.
- The insert SQL should eventually be parameterized or moved behind a helper routine.
- Generic control names make this form harder to maintain than it needs to be.

Form_CloseMessageBox

- The close message itself is fine.
- ResetSnapshotsTable should not have its own separate version of snapshot reset logic.
- This routine also uses SetWarnings False without a safe cleanup path.
- Be careful with deleting snapshot rows during close. Confirm that is still intentional.

Form_ShowUsersForm

- This is one of the cleaner files.
- It already has Option Explicit.
- The SELECT/WHERE/ORDER constants make the code easy to read.
- This is a good simple example of how the smaller forms should look.

Form_DynamicExportForm

- Small wrapper. Nothing major.
- Fix typo OuputDir to OutputDir when touched.
- Use a named control instead of Text1 if the form is updated.

Form_BranchMainForm

- Very small wrapper around ImpCalc.CalculateAllImpedances.
- Add Option Explicit and rename Command1 if the form is edited.
- No major build-stability concern found in this class object.

Potential way to do this

1. Back up the database and pick one known build case that the team can use to test the code.
2. Add Option Explicit to the class objects and compile. Fix only the compile issues first.
3. Add a shared ExecuteSql helper and start replacing SetWarnings False / RunSQL in the build path.
4. Pull snapshot update/reset logic into one helper routine.
5. Add simple build-stage logging around the ModelBuildForm path so a failed build says where it failed.
6. Refactor btnBuildFiles into smaller routines without changing the actual writer modules yet.
7. Replace the queue arrays with a queue item design or temp table.
8. Move authorized users/password out of VBA and into a table/config. (This one is optional)

9. Then review the standard modules, because that is where the bigger staging-table and file-bloat behavior probably lives.

Quick note for the later Modules review

The class objects show the setup and control problems, but they probably do not contain the full bloat root cause. The modules will be reviewed next for make-table queries, append/delete cycles, temporary/staging tables, and any build process that creates and deletes large objects. That is probably where the actual file growth and halfway-build failures will be.